

Project-First Roadmap to Become an Employable AI Engineer

1. Your starting point

You are not starting from zero.

You already have:

- Python fundamentals
- University-level data structures and algorithms
- Git and GitHub experience
- Basic Linux knowledge
- Flask and basic React
- Linear algebra, calculus and probability foundations
- Basic machine-learning knowledge
- Experience building a Flask/React AI chatbot
- A strong preference for understanding how algorithms work internally

Your main gaps are:

- Practical NumPy and Pandas
- Real data cleaning and preprocessing
- Applied statistics
- Building ML algorithms from scratch
- PyTorch and deep learning
- Model evaluation and debugging
- Production-quality Python
- SQL and data pipelines
- Deployment, MLOps and cloud infrastructure
- Designing complete AI systems

Your destination is not:

"I completed several AI courses."

Your destination is:

"I can take an unclear problem, prepare the data, create a baseline, implement and train a model, evaluate it properly, expose it through an API, deploy it, monitor it and explain every important decision."

2. The learning method

Every major project will have three layers.

Layer 1: Under the hood

Implement the mathematical core yourself.

Examples:

- Linear regression with NumPy
- Logistic regression with NumPy
- Decision-tree splitting
- Backpropagation
- Convolution
- Self-attention
- Cosine similarity
- Basic retrieval

The purpose is understanding, not recreating an entire professional library.

Layer 2: Professional implementation

Rebuild the solution using industry tools:

- NumPy
- Pandas
- scikit-learn
- PyTorch
- Hugging Face
- FastAPI
- PostgreSQL
- Docker
- MLflow
- DVC
- Qdrant or FAISS

Layer 3: Engineering and delivery

Turn the model into something usable:

- Clean project structure
- Configuration files
- Logging
- Unit tests

- API or application
- Docker container
- Reproducible training
- Evaluation report
- Demo
- GitHub documentation

The complete loop is:

```
Problem
↓
Data investigation
↓
Simple baseline
↓
Core algorithm from scratch
↓
Professional implementation
↓
Evaluation and error analysis
↓
API or application
↓
Testing and deployment
↓
Documentation and reflection
```

3. What counts as a completed project?

A project is not complete because the notebook runs.

Every portfolio project should eventually include:

1. A clear problem statement
2. A documented dataset
3. A simple baseline
4. Appropriate evaluation metrics
5. At least one experiment
6. Error analysis
7. Reproducible setup instructions
8. Clean Python modules instead of only notebooks
9. Tests for important functions
10. A README containing results and limitations
11. A demo, API or command-line interface

12. Evidence that you understand the implementation

Early projects do not need every production feature. The requirements grow as you progress.

Projects 1–4

Require:

- README
- Tests
- Clean Python modules
- Baseline and evaluation

Projects 5–10

Also require:

- API or interactive demo
- Docker
- Logging
- Configuration management
- GitHub Actions

Projects 11 onward

Also require:

- Experiment tracking
- Data or model versioning
- Monitoring
- Architecture diagram
- Evaluation dataset
- Deployment

Phase 0 — Build Your AI Engineering Environment

Estimated duration: 1–2 weeks

This is not a portfolio project. It is the template that all later projects will use.

Project 0: Reusable AI Project Template

Create a repository containing:

```
ai-project-template/  
├── src/  
│   └── project_name/  
├── tests/  
├── notebooks/  
├── data/  
├── models/  
├── reports/  
├── configs/  
├── scripts/  
├── Dockerfile  
├── pyproject.toml  
├── README.md  
└── .github/workflows/
```

Implement:

- Virtual environment and dependency management
- `src`-based Python package
- Type hints
- `pytest`
- Formatting and linting
- Logging
- Configuration file
- Pre-commit hooks
- Basic GitHub Actions workflow
- Simple Dockerfile

What you learn

- Python packaging
- Imports and modules
- Dependency management
- Testing
- Reproducibility
- Continuous integration
- Project organization

Completion gate

You should be able to clone the repository on another machine and run:

```
pytest
```

without manually fixing paths or dependencies.

Phase 1 — Data and Classical Machine Learning

Estimated duration: 3–4 months

The goal is to understand the complete tabular machine-learning workflow before deep learning.

Project 1: Data Quality Inspector

Build

Create a Python tool that accepts a CSV file and generates a data-quality report.

The report should identify:

- Missing values
- Duplicate rows
- Incorrect data types
- Constant columns
- Suspicious values
- Outliers
- Class imbalance
- Correlations
- Possible target leakage

Under the hood

Implement some of these yourself before using Pandas:

- Mean
- Median
- Variance
- Standard deviation
- Quantiles
- Z-score
- Min-max scaling
- Standardization
- Train/test splitting

Then vectorize the implementations using NumPy.

Professional version

Use:

- NumPy
- Pandas
- Matplotlib
- SQL
- A validation schema
- Automated HTML or Markdown report

Engineering layer

Add:

- Command-line interface
- Unit tests
- Logging
- Configuration
- Sample datasets
- A report generated by the application

What you learn

- NumPy
 - Pandas
 - Applied statistics
 - Data leakage
 - Exploratory data analysis
 - Data validation
 - SQL basics
-

Project 2: Rental Price Predictor

This can use a public housing or rental dataset.

Under the hood

Implement linear regression using:

- The prediction equation
- Mean-squared error
- Gradient descent
- Feature scaling
- Vectorized matrix operations
- Learning-rate experimentation

Optionally implement the normal-equation solution and compare it with gradient descent.

Professional version

Use a scikit-learn pipeline containing:

- Missing-value handling
- Numerical scaling
- Categorical encoding
- Linear regression
- Regularized regression
- Cross-validation

Compare:

- Mean baseline
- Your NumPy model
- Scikit-learn linear regression
- Ridge or Lasso regression
- A tree-based model

Important investigations

- What happens when the learning rate is too large?
- Which features have the strongest effect?
- How do outliers affect the model?
- Does regularization improve test performance?
- Are the errors larger for particular categories?

Ship

Create:

- A prediction CLI
- A simple web form or FastAPI endpoint
- Tests for preprocessing and prediction
- A model card describing limitations

Mathematics learned naturally

- Vectors and matrices
- Derivatives
- Gradients
- Loss functions
- Regularization
- Bias and variance

Project 3: Customer Churn Classifier

Under the hood

Implement logistic regression yourself:

- Sigmoid
- Binary cross-entropy
- Gradient calculation
- Gradient descent
- Probability prediction
- Classification threshold

Verify your gradients using numerical gradient checking.

Professional version

Compare:

- Majority-class baseline
- Logistic regression
- K-nearest neighbours
- Decision tree
- Random forest
- Gradient boosting

Evaluation

Do not report only accuracy.

Use:

- Confusion matrix
- Precision
- Recall
- F1 score
- ROC-AUC
- Precision-recall curve
- Probability calibration

Experiment with different thresholds based on business costs.

Engineering layer

- Reusable preprocessing pipeline
- Configuration-driven training
- Saved model artifact
- Batch prediction script

- API endpoint
- Automated tests

What you learn

- Classification
 - Probabilities
 - Thresholds
 - Imbalanced datasets
 - Metrics
 - Model calibration
 - Business-oriented evaluation
-

Project 4: Similarity and Clustering Laboratory

Build one application containing several distance-based algorithms.

A good application would be an image colour compressor or customer-segmentation tool.

Under the hood

Implement:

- Euclidean distance
- Manhattan distance
- K-nearest neighbours
- K-means clustering
- K-means++ initialization
- Principal component analysis using matrix operations

Investigate

- Why scaling changes distance-based models
- How different values of `k` affect results
- Sensitivity to initialization
- Curse of dimensionality
- Cluster quality
- PCA explained variance

Professional version

Compare your implementations with scikit-learn.

Ship

Provide:

- Visualizations
 - Before-and-after comparisons
 - Runtime comparison
 - Unit tests
 - Complexity discussion
-

Project 5: Decision Tree From Scratch

You have already implemented a decision tree in university. Do not discard that work.

Turn it into a professional public project.

Improve your implementation

Add:

- Entropy
- Gini impurity
- Information gain
- Continuous feature splitting
- Maximum depth
- Minimum samples per split
- Prediction for unseen examples
- Tree visualization
- Basic pruning

Compare against scikit-learn

Investigate:

- Overfitting
- Tree depth
- Entropy versus Gini
- Training and validation performance
- Feature importance
- Pruned versus unpruned trees

Engineering layer

Convert the implementation into a scikit-learn-style class:

```
model.fit(X, y)
model.predict(X)
model.score(X, y)
```

Add:

- Type hints
- Tests
- Documentation
- Example notebook
- Graph visualization

This project demonstrates that you understand one of the most important classical ML algorithms internally.

Project 6: Production Tabular ML System

This is your first major flagship project.

Choose one meaningful tabular problem:

- Customer churn
- Energy demand
- Delivery-time prediction
- Equipment failure
- Transaction anomaly detection
- Student-course outcome analysis using an appropriate public dataset

Complete system

Build:

```
Raw data
  ↓
Validation
  ↓
Preprocessing
  ↓
Feature engineering
  ↓
Training
  ↓
Evaluation
  ↓
```

```
graph TD;
    A[Saved model] --> B[FastAPI service];
    B --> C[Docker container];
```

Required features

- SQL data ingestion
- Train/validation/test separation
- Baseline model
- At least three model families
- Hyperparameter search
- Cross-validation
- Error analysis
- Reproducible training command
- Prediction API
- Input validation
- API tests
- Docker
- GitHub Actions
- Logging

Portfolio evidence

Your README should explain:

- The real-world problem
- How the data flows through the system
- Why you selected the final model
- Important mistakes you discovered
- What you would monitor after deployment
- Limitations and ethical considerations

Employability checkpoint

After Project 6, begin applying for:

- Data-science working-student positions
- Junior data roles
- ML internships
- Research-assistant opportunities
- Python/backend positions involving data

Do not wait until the entire roadmap is complete.

Phase 2 — Deep Learning From the Inside Out

Estimated duration: 3–4 months

Project 7: Automatic Differentiation Engine

Build a small scalar automatic-differentiation library.

Your engine should support:

- Addition
- Multiplication
- Powers
- Basic activation functions
- Computational graphs
- Topological sorting
- Backward propagation
- Gradient accumulation

Example:

```
x = Value(2.0)
w = Value(3.0)
b = Value(1.0)

y = w * x + b
y.backward()
```

What you learn

- Computational graphs
- Chain rule
- Backpropagation
- Gradient flow
- Why gradients accumulate
- What `loss.backward()` actually does

Validation

Compare calculated gradients against:

- Hand calculations
- Finite differences
- PyTorch autograd

This project does not need a user interface. Its value is in the implementation and explanation.

Project 8: Neural Network From Scratch for Handwritten Digits

Start with the handwritten-digit idea you have already discussed.

Begin with digits 1 and 2, using multiple writing styles. Later expand to all ten digits.

Data stage

- Collect or use handwritten images
- Create train, validation and test sets
- Inspect class distribution
- Normalize image values
- Visualize examples
- Create deliberately difficult examples

Under the hood

Using only NumPy, implement:

- Dense layer
- ReLU
- Softmax
- Cross-entropy
- Forward propagation
- Backpropagation
- Mini-batch gradient descent
- Parameter initialization
- Training loop

Experiments

- One hidden layer versus two
- Different learning rates
- Different hidden dimensions
- ReLU versus sigmoid
- Overfitting a tiny dataset
- With and without normalization

Critical debugging exercises

- Intentionally break a gradient
- Inspect exploding or vanishing values
- Verify tensor shapes
- Overfit ten examples

- Compare analytical and numerical gradients

Ship

Create an application where a user can draw a digit and receive:

- Predicted class
- Confidence
- Probability distribution

Project 9: Rebuild the Digit Classifier in PyTorch

Reimplement Project 8 using:

- Dataset
- DataLoader
- nn.Module
- Optimizers
- Automatic differentiation
- GPU support
- Checkpoints
- Training and evaluation modes

Compare

Document exactly how these correspond:

Your implementation	PyTorch
Dense layer	<code>nn.Linear</code>
Manual ReLU	<code>nn.ReLU</code>
Manual gradients	Autograd
Parameter update	Optimizer
Mini-batch loop	<code>DataLoader</code>
Saved arrays	<code>state_dict</code>

Engineering layer

- Configuration-driven experiments
- Reproducible random seeds
- Checkpoint saving
- Resume training
- TensorBoard or MLflow logging

- Unit tests for dataset and model output
- Dockerized inference API

The point is not only better accuracy. The point is understanding what PyTorch automates.

Project 10: Convolution and Image Classification

Under the hood

Implement:

- Image kernels
- Padding
- Stride
- Two-dimensional convolution forward pass
- Max pooling
- Output-size calculation

You do not need to build a complete industrial CNN library. Implement enough to understand how convolution transforms an image.

Professional version

Train a PyTorch CNN on a meaningful image dataset.

Possible applications:

- Waste classification
- Plant-type classification
- Surface-defect classification
- Traffic-sign recognition
- Food classification

Compare:

- Simple dense neural network
- Small CNN
- Transfer-learning model

Investigate

- Data augmentation
- Class imbalance
- Misclassified images
- Learning curves
- Frozen versus unfrozen pretrained layers
- Inference speed

- Model size

Ship

- Image upload interface
 - Prediction API
 - Confidence output
 - Docker container
 - Error gallery
 - Architecture diagram
-

Project 11: Deep-Learning Experiment Laboratory

Take Project 10 and systematically investigate model training.

Run controlled experiments on:

- Optimizer choice
- Learning rate
- Batch size
- Weight decay
- Dropout
- Batch normalization
- Data augmentation
- Scheduler
- Random seeds

Track:

- Training loss
- Validation loss
- Accuracy
- F1 score
- Runtime
- Model size
- Experiment configuration

Deliverable

Write an experiment report answering:

- Which change helped?
- Which change only improved training accuracy?
- Which results were unstable across seeds?
- Which model would you deploy?
- What trade-off did you make?

This teaches experimental discipline rather than random hyperparameter changing.

Phase 3 — NLP and Transformers

Estimated duration: 4–5 months

Project 12: Text Classifier From Classical NLP to Transformers

Build a classifier for:

- Support-ticket routing
- News categories
- Intent classification
- Spam detection
- Review sentiment

Under the hood

Implement:

- Text cleaning
- Token counts
- Bag of words
- Term frequency
- Inverse document frequency
- TF-IDF
- Multinomial Naive Bayes

Professional comparison

Compare:

1. Majority baseline
2. Naive Bayes
3. Logistic regression with TF-IDF
4. A pretrained transformer

Evaluation

Investigate:

- Rare classes
- Confusing classes
- Vocabulary size

- Unknown words
- Short versus long texts
- Confidence calibration

Ship

Create a text-classification API and simple frontend.

Project 13: Tokenizer and Self-Attention From Scratch

This is the foundation for understanding transformers.

Part A: Tokenization

Implement a small byte-pair encoding tokenizer:

- Initial vocabulary
- Pair frequencies
- Merge rules
- Encoding
- Decoding
- Unknown tokens

Compare your tokenizer with an existing tokenizer.

Part B: Attention

Using NumPy or basic PyTorch tensor operations, implement:

- Token embeddings
- Positional information
- Query, key and value projections
- Scaled dot-product attention
- Causal masking
- Multi-head attention
- Residual connection
- Layer normalization
- Feed-forward block

Required visualizations

Visualize:

- Attention matrices
- Causal masks
- Token relationships

- Tensor shapes after each operation

Completion gate

You should be able to explain:

- Why the dot product measures token relevance
 - Why division by the square root of key dimension is used
 - Why masking is necessary
 - What different heads can represent
 - Why residual connections help
-

Project 14: Tiny Transformer Language Model

Build a small decoder-only transformer.

This is an educational model, not an attempt to compete with foundation models.

Implement

- Tokenization
- Embedding layer
- Positional representation
- Transformer blocks
- Causal self-attention
- Language-model output head
- Cross-entropy training
- Autoregressive generation
- Temperature
- Top-k sampling

Train on

- A small public-domain text corpus
- Code from a small permitted dataset
- A carefully prepared domain-specific corpus

Evaluate

- Training and validation loss
- Perplexity
- Generated samples
- Repetition
- Memorization
- Context-length behaviour

Engineering questions

- How much memory does training require?
 - Which operations dominate runtime?
 - What changes with sequence length?
 - How does batch size affect training?
 - What causes poor generation?
-

Project 15: Fine-Tune a Pretrained Transformer

Use Hugging Face to fine-tune an encoder or small generative model.

A strong project would be multilingual German-English support-ticket routing or document classification.

Implement

- Dataset preparation
- Tokenization
- Training and evaluation loop
- Checkpointing
- Metric calculation
- Error analysis
- Model publishing or packaging

Compare

- Classical TF-IDF model
- Frozen pretrained embeddings
- Fine-tuned transformer
- Parameter-efficient fine-tuning where appropriate

Important lesson

The transformer should not automatically be declared the winner.

Compare:

- Accuracy
 - Latency
 - Model size
 - Training cost
 - Explainability
 - Maintenance complexity
-

Project 16: Semantic Search Engine

Build a search engine for a collection of documents.

Under the hood

Implement:

- Text chunking
- Embedding generation
- Cosine similarity
- Brute-force nearest-neighbour search
- Ranking
- Recall evaluation

Professional version

Add:

- FAISS or Qdrant
- Metadata filters
- Dense retrieval
- Keyword retrieval
- Hybrid retrieval
- Reranking

Create an evaluation set

Write approximately 50–100 realistic queries with known relevant documents.

Measure:

- Recall@k
- Precision@k
- Mean reciprocal rank
- Latency

Ship

Create:

- Search API
 - Search interface
 - Highlighted results
 - Filters
 - Evaluation report
-

Project 17: Production-Quality RAG System

This should become one of your strongest portfolio projects.

A relevant idea for you is:

A multilingual assistant for public university regulations, module handbooks and student-information documents.

Use only documents that can legally and safely be included.

Build the system in stages

Version 1

- Load documents
- Extract text
- Split into chunks
- Generate embeddings
- Retrieve similar chunks
- Send context to an LLM
- Return an answer with sources

Version 2

- Better document parsing
- Metadata
- Hybrid retrieval
- Query rewriting
- Reranking
- Citation validation
- Conversation history
- Caching

Version 3

- Evaluation dataset
- Retrieval metrics
- Answer-quality evaluation
- Hallucination checks
- Prompt-injection tests
- Access controls
- Cost and latency tracking
- User-feedback collection

Required comparisons

Compare:

- Different chunk sizes
- Overlap settings
- Embedding models
- Dense versus hybrid retrieval
- With versus without reranking
- Different numbers of retrieved chunks
- Grounded versus ungrounded answers

Architecture

```
graph TD; Documents --> Parsing_and_cleaning[Parsing and cleaning]; Parsing_and_cleaning --> Chunking_and_metadata[Chunking and metadata]; Chunking_and_metadata --> Embedding_and_indexing[Embedding and indexing]; Embedding_and_indexing --> Retrieval; Retrieval --> Reranking; Reranking --> LLM_generation[LLM generation]; LLM_generation --> Citation_validation[Citation validation]; Citation_validation --> API_and_user_interface[API and user interface];
```

Documents
↓
Parsing and cleaning
↓
Chunking and metadata
↓
Embedding and indexing
↓
Retrieval
↓
Reranking
↓
LLM generation
↓
Citation validation
↓
API and user interface

What makes this project employable

The impressive part is not that it “chats with PDFs.”

The impressive part is:

- Evaluation
- Traceability
- Retrieval quality
- Security awareness
- Testing
- Deployment
- Monitoring
- Honest discussion of failure cases

Project 18: Tool-Using AI Workflow

Do not immediately hide everything behind an agent framework.

First build a controlled workflow yourself.

Possible system:

An AI research assistant that searches a local document collection, performs calculations, queries a database and creates a structured report.

Implement manually

- Tool definitions
- Input schemas
- Tool selection
- State
- Execution loop
- Maximum-step limit
- Error handling
- Human approval for important actions
- Structured output
- Logs and traces

Represent the workflow as a state machine.

Only after understanding it should you reproduce the workflow using one agent framework.

Evaluate

- Tool-selection accuracy
- Task-completion rate
- Invalid tool calls
- Number of steps
- Latency
- Cost
- Recovery from tool failures

This prevents “agent demos” that look impressive but cannot be tested reliably.

Phase 4 — MLOps and Production Engineering

Estimated duration: 2–3 months

You should not postpone all engineering until this phase. Earlier projects already introduce APIs, tests and Docker.

This phase connects everything into a professional workflow.

Project 19: Reproducible Training Platform

Take Project 6, 10, 15 or 17 and build a reproducible training system around it.

Add

- Data versioning
- Training pipeline
- Configuration files
- Experiment tracking
- Model registry
- Model versioning
- Artifact storage
- Automated evaluation
- Model promotion rules
- CI pipeline

Example workflow

```
New code or configuration
↓
Tests
↓
Training pipeline
↓
Evaluation
↓
Compare against current model
↓
Register candidate
↓
Deploy only if requirements pass
```

Tools

Use:

- Git
- DVC
- MLflow

- GitHub Actions
- Object storage
- PostgreSQL where appropriate

Required result

Another developer should be able to reproduce the selected model from:

- A Git commit
 - A data version
 - A configuration file
 - A training command
-

Project 20: Deployed and Monitored AI Service

This is your production capstone.

Take your strongest model and deploy it on one cloud provider.

Do not try to learn AWS, Azure and Google Cloud simultaneously. Choose one.

System components

- FastAPI backend
- Docker
- Database
- Model artifact storage
- Authentication
- Structured logs
- Health endpoint
- Metrics
- Error tracking
- Rate limiting
- CI/CD
- Secret management

Model monitoring

Track:

- Request count
- Prediction latency
- Errors
- Input distributions
- Missing values
- Prediction distributions

- Confidence
- Data drift
- Model quality when labels arrive

Simulate production problems

Test what happens when:

- Input fields are missing
- Data types change
- A category is unseen
- Requests arrive concurrently
- The model file is missing
- The database becomes unavailable
- Input distribution changes
- A new model performs worse

Final deliverables

- Live deployment
 - Architecture diagram
 - Monitoring dashboard
 - Load-test results
 - Incident-response notes
 - Reproducible deployment instructions
 - Technical retrospective
-

Phase 5 — Specialization Capstone

Estimated duration: 2–3 months

After the common core, choose one specialization. Do not try to complete every branch.

Option A: Computer Vision

Build an object-detection, segmentation or industrial-inspection system.

Possible project:

Detect and classify surface defects from manufacturing images.

Include:

- Data annotation
- Augmentation

- Transfer learning
- Object detection or segmentation
- Error analysis by defect type
- Inference optimization
- API deployment

Option B: NLP and LLM Systems

Expand the university-document assistant into a production document-intelligence platform.

Include:

- Multiple document formats
- Structured extraction
- Hybrid search
- Reranking
- RAG evaluation
- User roles
- Feedback
- Monitoring
- Multilingual support

Option C: Audio AI

This connects well with your Audio & AI elective.

Build:

An environmental-sound or spoken-command classification system.

Include:

- Waveforms
- Spectrograms
- Feature extraction
- CNN or transformer model
- Noise augmentation
- Real-time inference
- Audio API
- Latency evaluation

Option D: Recommendation Systems

Build:

A recommendation service combining collaborative and content-based filtering.

Include:

- Popularity baseline
- Matrix factorization
- Embedding-based retrieval
- Ranking
- Cold-start handling
- Offline evaluation
- Recommendation API

Your specialization capstone should solve a larger problem than the earlier learning projects and should look like work produced by a small engineering team.

4. Recommended GitHub Structure

Do not create twenty tiny repositories with copied notebooks.

Use approximately:

- One foundations repository
- Four to six medium project repositories
- Three flagship repositories

Example:

```
ai-foundations/  
├─ linear-regression  
├─ logistic-regression  
├─ knn-kmeans  
├─ decision-tree  
├─ autograd  
├─ neural-network  
└─ attention
```

Separate flagship repositories:

```
production-tabular-ml/  
handwritten-digit-system/  
production-rag-assistant/  
mlops-model-platform/  
specialization-capstone/
```

Pin your best three projects on GitHub.

5. README Template for Every Flagship Project

```
# Project title

## Problem

## Why this matters

## System architecture

## Dataset

## Baseline

## From-scratch implementation

## Professional implementation

## Experiments

## Evaluation

## Error analysis

## API or application

## Testing

## Deployment

## Limitations

## What I learned

## Reproducing the project
```

Also include:

- Screenshots
- Architecture diagram
- Metric tables
- Example API requests
- Short demonstration video
- Clear installation commands

6. Weekly Learning System

Assuming approximately 10 hours per week:

50% — Building

Write the implementation and make the system work.

20% — Just-in-time theory

Study only the concepts blocking the current project.

Examples:

- Gradient descent while implementing regression
- Probability while building Naive Bayes
- Chain rule while implementing backpropagation
- Matrix operations while implementing attention

15% — Evaluation and debugging

- Design experiments
- Inspect incorrect predictions
- Check gradients
- Test edge cases
- Compare against baselines

15% — Engineering and documentation

- Refactor
- Add tests
- Improve README
- Create diagrams
- Record results
- Commit changes

A typical project week could look like:

Session 1

Define the problem, dataset, metric and baseline.

Sessions 2-3

Implement the core mechanism yourself.

Session 4

Rebuild using professional libraries.

Session 5

Run experiments and analyse errors.

Session 6

Refactor, test, document and publish.

7. Just-in-Time Mathematics

Do not stop projects for six months to “finish all mathematics.”

Learn mathematics when a project creates a need for it.

Project	Mathematics
Linear regression	Vectors, derivatives, gradients
Logistic regression	Probability, logarithms, likelihood
PCA	Eigenvectors, projections, variance
Decision trees	Entropy, probability
Neural networks	Chain rule, matrix calculus
CNNs	Linear operations, spatial dimensions
Attention	Matrix multiplication, softmax, similarity
Evaluation	Statistics, confidence intervals
Monitoring	Distributions, drift and hypothesis testing

For every important algorithm:

1. Write the formula.
2. Calculate one tiny example manually.
3. Write pseudocode.
4. Implement a loop-based version.

5. Vectorize it.
 6. Compare against a library.
 7. Explain time and space complexity.
-

8. Skills to Maintain Alongside the Projects

Python

Continue improving:

- Classes and dataclasses
- Iterators and generators
- Decorators
- Context managers
- Type hints
- Exceptions
- Async programming
- Packaging
- Profiling
- Memory usage

SQL

Practise:

- Joins
- Aggregation
- Subqueries
- Common table expressions
- Window functions
- Indexes
- Query plans

Aim for two or three SQL problems each week.

Data structures and algorithms

Maintain your university knowledge rather than making it your main focus.

Practise approximately two good problems per week covering:

- Arrays and strings
- Hash maps
- Trees

- Graphs
- Heaps
- Recursion
- Dynamic programming
- Complexity

Communication

For each project, practise explaining:

- The problem
 - The baseline
 - The chosen metric
 - Why the model was selected
 - One failure
 - One trade-off
 - What you would improve
-

9. What Not to Do

Avoid:

- Watching complete courses before building anything
- Copying tutorial projects line by line
- Publishing only Jupyter notebooks
- Using accuracy for every problem
- Calling an API and presenting it as a complete AI project
- Learning five agent frameworks
- Starting Kubernetes before you can deploy one Docker container
- Training enormous models
- Reimplementing full production libraries
- Building ten versions of the same chatbot
- Spending months making a frontend while the model is poorly evaluated
- Hiding weak retrieval behind impressive-looking LLM responses
- Waiting until graduation before applying

The purpose of “from scratch” is to expose the important mechanisms—not to replace PyTorch, scikit-learn or vector databases.

10. Suggested Timeline

This timeline assumes university studies alongside approximately 8–12 hours of project work per week.

Months	Main work
1	Project template and Data Quality Inspector
2–3	Linear regression, logistic regression and clustering
4	Decision tree and production tabular system
5	Autograd engine
6	NumPy handwritten-digit neural network
7	PyTorch digit system
8–9	CNN and deep-learning experiments
10	Classical NLP text classifier
11	Tokenizer and attention
12	Tiny transformer
13	Transformer fine-tuning
14	Semantic search
15–16	Production RAG
17	Tool-using workflow
18	Reproducible ML pipeline
19	Deployment and monitoring
20–22	Specialization capstone and applications

During semester breaks, projects may move faster. During examinations, reduce the workload rather than abandoning the system.

11. Employability Milestones

After Project 6

You should be able to:

- Clean tabular data
- Build baselines
- Train classical models
- Evaluate them correctly
- Build a prediction API
- Use Docker and tests

Start applying for working-student and internship roles.

After Project 10

You should also be able to:

- Explain backpropagation
- Train PyTorch models
- Debug training
- Build computer-vision systems
- Use transfer learning
- Deploy neural-network inference

You are increasingly competitive for ML and deep-learning internships.

After Project 17

You should also be able to:

- Understand transformer internals
- Fine-tune pretrained models
- Build semantic retrieval
- Build and evaluate RAG systems
- Integrate models into applications

You can target AI engineer, applied AI and LLM application roles in addition to ML roles.

After Projects 19–20

You should be able to:

- Reproduce experiments
- Track models and datasets
- Deploy cloud services
- Build CI/CD pipelines
- Monitor production behaviour
- Discuss production failures

This is the level where your portfolio begins to demonstrate genuine AI engineering rather than only model experimentation.

12. Your Final Portfolio

By the end, aim for:

Three flagship systems

1. **Production tabular ML service**
2. **Deep-learning or specialization system**
3. **Production RAG or NLP system**

Four to six supporting projects

Examples:

- Data Quality Inspector
- Decision Tree From Scratch
- Autograd Engine
- NumPy Neural Network
- Transformer From Scratch
- Semantic Search Engine

Evidence of engineering skill

Across the portfolio:

- Python
- NumPy and Pandas
- SQL
- scikit-learn
- PyTorch
- Hugging Face
- FastAPI
- Docker
- Tests
- CI/CD
- MLflow
- DVC
- Cloud deployment
- Monitoring

Evidence of understanding

Your projects should prove that you can:

- Derive core algorithms
- Implement them
- Compare them with professional libraries
- Select meaningful metrics
- Analyse errors
- Design experiments

- Make engineering trade-offs
- Explain limitations
- Deliver working systems

That combination—not the number of tutorials or certificates—is what this roadmap is designed to build.